

Chapter 9 — Binary Search

0. What You Will Learn in This Chapter

Binary Search is one of the most important coding interview patterns.

Most beginners think Binary Search only means:

Find a target in a sorted array.

But in interviews, Binary Search appears in many forms:

- exact target search
- insertion position
- first occurrence
- last occurrence
- lower bound
- upper bound
- first true / last false
- rotated sorted array search
- search in 2D matrix
- find minimum in rotated sorted array
- find peak element
- single element in sorted array
- square root / answer boundary problems

This chapter focuses on **Binary Search on sorted structures and boundaries**.

The next chapter will separately cover **Binary Search on Answer**, which is the advanced form used in problems like:

- Koko Eating Bananas
- Ship Packages Within D Days
- Split Array Largest Sum
- Magnetic Force Between Two Balls

By the end of this chapter, you should understand:

- What binary search is
- Why binary search works
- When to use binary search
- When not to use binary search
- How to avoid infinite loops
- How to handle duplicates
- How to implement lower bound and upper bound
- How to search first/last occurrence
- How to search rotated arrays
- How to search matrices
- How to solve special sorted-structure problems
- Common binary search edge cases
- Common off-by-one mistakes

Core LeetCode problems covered in this chapter:

- 1.1. Binary Search
- 2.2. Search Insert Position
- 3.3. Find First and Last Position of Element in Sorted Array
- 4.4. Find Smallest Letter Greater Than Target
- 5.5. First Bad Version
- 6.6. Search in Rotated Sorted Array
- 7.7. Find Minimum in Rotated Sorted Array

- 8.8. Search a 2D Matrix
 - 9.9. Search a 2D Matrix II
 - 10.10. Find Peak Element
 - 11.11. Single Element in a Sorted Array
 - 12.12. Sqrt(x)
 - 13.13. Valid Perfect Square
 - 14.14. H-Index II
 - 15.15. Peak Index in a Mountain Array
-

1. What Is Binary Search?

Binary Search is a technique that repeatedly cuts the search space in half. It works when the search space has some kind of order or monotonic property. The simplest example:

```
nums = {-1, 0, 3, 5, 9, 12}
target = 9
```

Instead of checking every element, we check the middle.
If middle is too small, search right half.
If middle is too large, search left half.
If middle equals target, we found it.

Simple Binary Search Idea

```
left = 0
right = n - 1

while left <= right:
    mid = middle index

    if nums[mid] == target:
        found
    else if nums[mid] < target:
        search right half
    else:
        search left half
```

Each step removes about half the elements.
That is why binary search is $O(\log n)$.

2. Why Binary Search Exists

Binary Search exists because sorted order gives useful information. Suppose:

```
nums = {1, 3, 5, 7, 9, 11}
target = 9
```

If we check middle:

```
nums[mid] = 5
```

Since:

```
5 < 9
```

everything left of 5 is also less than or equal to 5.

So none of those values can be 9.

We can safely discard the left half.

That is the key:

Binary search works when one comparison lets you eliminate half the remaining search space.

3. Core Need / Why Use Binary Search Over Other Patterns?

Binary Search is useful when the problem asks you to search in an ordered space.

3.1 Why Not Linear Search?

Linear search checks every element.

Time:

```
O(n)
```

Binary search checks logarithmically many elements.

Time:

```
O(log n)
```

For $n = 1,000,000$:

- Linear search may check 1,000,000 elements.
- Binary search checks about 20 elements.

3.2 Why Not Hash Map?

Hash map is good for exact lookup.

But binary search is better when you need:

- insertion index
- first occurrence
- last occurrence
- lower bound
- upper bound
- nearest greater element
- sorted boundary
- monotonic transition point

Example:

```
nums = {1, 2, 2, 2, 3}
```

```
target = 2
```

Hash map can say 2 exists.

Binary search can find:

first index = 1
last index = 3

3.3 Why Not Sorting + Search?

If input is already sorted, binary search is natural.

But if input is unsorted and original order matters, sorting can be wrong.

Example:

Stock prices:

prices = {7, 1, 5, 3, 6, 4}

You cannot sort because day order matters.

3.4 Why Not Two Pointers?

Two pointers is great for pair problems in sorted arrays.

Binary search is better when searching for:

- one target
- one boundary
- first true
- first position satisfying condition

4. Core Requirement for Binary Search

Binary Search needs one of these:

4.1 Sorted Data

Example:

nums = {1, 3, 5, 7, 9}

You can search a value.

4.2 Monotonic Boolean Condition

Example:

false false false true true true

You can binary search for the first true.

Or:

true true true false false false

You can binary search for the last true.

4.3 Ability to Eliminate Half

At each middle point, you must be able to decide:

answer is definitely not in one half

If you cannot eliminate a half, binary search is not valid.

5. When to Use Binary Search

Use binary search when you see these clues.

5.1 Sorted Array Search

Problem says:

- sorted array
- ascending order
- non-decreasing order
- find target
- return index

Examples:

- Binary Search
 - Search Insert Position
 - Find First and Last Position
-

5.2 Boundary Search

Problem asks:

- first occurrence
- last occurrence
- first index where condition holds
- insertion position
- smallest value greater than target
- lower bound
- upper bound

Examples:

- Search Insert Position
 - Find First and Last Position
 - Find Smallest Letter Greater Than Target
 - First Bad Version
-

5.3 Rotated Sorted Array

Problem says:

- sorted array rotated
- search in rotated array
- find minimum in rotated array

Examples:

- Search in Rotated Sorted Array
 - Find Minimum in Rotated Sorted Array
-

5.4 Matrix With Sorted Structure

Examples:

- Search a 2D Matrix
 - Search a 2D Matrix II
-

5.5 Special Sorted Pattern

Examples:

- Single Element in a Sorted Array
 - Find Peak Element
 - H-Index II
 - Sqrt(x)
 - Valid Perfect Square
 - Peak Index in a Mountain Array
-

6. When NOT to Use Binary Search

6.1 Data Is Not Sorted and Has No Monotonic Property

Example:

nums = {4, 1, 7, 2, 9}

You cannot binary search for 7 unless the array is sorted or has another searchable structure.

6.2 You Need to Check Every Element

Example:

Find sum of all values.

Binary search does not help.

6.3 Sorting Would Destroy Meaning

Examples:

- stock prices
 - subarray problems
 - subsequence problems
 - time/order-dependent problems
-

6.4 Predicate Is Not Monotonic

Binary search needs a clean transition.

Good:

false false false true true true

Bad:

false true false true false

If condition is not monotonic, binary search gives invalid results.

7. Binary Search Mental Models

7.1 Guess and Eliminate

Binary search guesses the middle and eliminates half.

At each step:

2. Pick middle.
3. Compare.

4. Remove impossible half.

7.2 Boundary Between Bad and Good

Many binary search problems are boundary problems.

Example:

bad bad bad good good good

Find first good.

This is used in:

- First Bad Version
 - lower bound
 - minimum feasible answer, next chapter
-

7.3 Lower Bound

Lower bound means:

First index where value is greater than or equal to target.

Example:

nums = {1, 3, 3, 5, 8}

target = 3

Lower bound index:

1

For:

target = 4

Lower bound index:

3

because nums[3] = 5.

7.4 Upper Bound

Upper bound means:

First index where value is greater than target.

Example:

nums = {1, 3, 3, 5, 8}

target = 3

Upper bound index:

3

This is one position after the last 3.

7.5 Search Space Can Be Indices, Values, or Answers

Binary search does not always search an array.

It can search:

- indices
- values

- answer range
 - version numbers
 - row/column positions
 - mountain peak positions
-

8. C++ Tools Needed

8.1 Safe Middle Formula

Avoid:

```
int mid = (left + right) / 2;
```

because left + right can overflow.

Use:

```
int mid = left + (right - left) / 2;
```

8.2 lower_bound

```
auto it = lower_bound(nums.begin(), nums.end(), target);
```

Returns iterator to first element:

```
>= target
```

Convert to index:

```
int index = it - nums.begin();
```

8.3 upper_bound

```
auto it = upper_bound(nums.begin(), nums.end(), target);
```

Returns iterator to first element:

```
> target
```

8.4 binary_search

```
bool exists = binary_search(nums.begin(), nums.end(), target);
```

Returns only true/false.

In interviews, we usually write binary search manually because we need indices or boundaries.

8.5 long long

Use when:

- computing mid * mid
- searching large ranges
- adding large numbers

Example:

```
long long square = 1LL * mid * mid;
```

9. Binary Search Styles

There are two main styles.

9.1 Inclusive Range [left, right]

```
int left = 0;
int right = n - 1;

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (...) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}
```

Use for:

- exact search
- first/last occurrence with answer variable
- many answer-search problems

9.2 Half-Open Range [left, right)

```
int left = 0;
int right = n;

while (left < right) {
    int mid = left + (right - left) / 2;

    if (...) {
        left = mid + 1;
    } else {
        right = mid;
    }
}
```

Use for:

- lower bound
- upper bound
- insertion position

9.3 Do Not Mix Templates Randomly

Most binary search bugs come from mixing:

```
while (left <= right)
```

with:

```
right = mid
```

or:

```
while (left < right)
```

with:

```
right = mid - 1
```

Both styles work, but be consistent.

10. Core Templates

Template 1 — Exact Binary Search

Use when you need to find target or return -1.

```
int binarySearch(vector<int>& nums, int target) {
    int left = 0;
    int right = (int)nums.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}
```

Template 2 — Lower Bound

First index where:

```
nums[i] >= target
```

```

int lowerBound(vector<int>& nums, int target) {
    int left = 0;
    int right = nums.size();

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }

    return left;
}

```

Return can be nums.size().

Template 3 — Upper Bound

First index where:

```

nums[i] > target
int upperBound(vector<int>& nums, int target) {
    int left = 0;
    int right = nums.size();

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] <= target) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }

    return left;
}

```

Template 4 — First Occurrence

```

int firstOccurrence(vector<int>& nums, int target) {
    int left = 0;

```

```
int right = nums.size() - 1;
int answer = -1;

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (nums[mid] >= target) {
        if (nums[mid] == target) {
            answer = mid;
        }

        right = mid - 1;
    } else {
        left = mid + 1;
    }
}

return answer;
}
```

Template 5 — Last Occurrence

```
int lastOccurrence(vector<int>& nums, int target) {
    int left = 0;
    int right = nums.size() - 1;
    int answer = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] <= target) {
            if (nums[mid] == target) {
                answer = mid;
            }

            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return answer;
}
```

Template 6 — First True

For pattern:

```
false false false true true true
int firstTrue(int n) {
    int left = 0;
    int right = n - 1;
    int answer = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (condition(mid)) {
            answer = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }

    return answer;
}
```

Template 7 — Last True

For pattern:

```
true true true false false false
int lastTrue(int n) {
    int left = 0;
    int right = n - 1;
    int answer = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (condition(mid)) {
            answer = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
}
```

```
    return answer;
}
```

Template 8 — Rotated Sorted Array Search

```
int searchRotated(vector<int>& nums, int target) {
    int left = 0;
    int right = nums.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            return mid;
        }

        if (nums[left] <= nums[mid]) {
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }

    return -1;
}
```

Template 9 — Find Minimum in Rotated Sorted Array

```
int findMin(vector<int>& nums) {
    int left = 0;
    int right = nums.size() - 1;

    while (left < right) {
        int mid = left + (right - left) / 2;
```

```

        if (nums[mid] > nums[right]) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }

    return nums[left];
}

```

Template 10 — Search 2D Matrix as Flattened Array

```

bool searchMatrix(vector<vector<int>>& matrix, int target) {
    int rows = matrix.size();
    int cols = matrix[0].size();

    int left = 0;
    int right = rows * cols - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        int r = mid / cols;
        int c = mid % cols;

        if (matrix[r][c] == target) {
            return true;
        } else if (matrix[r][c] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return false;
}

```

11. Major Variations of Binary Search

Exact search	find target index	Binary Search
Lower bound	first $\geq$ target	Search Insert Position
Upper bound	first $>$ target	Next Greatest Letter
First occurrence	left boundary	Find First and Last Position
Last occurrence	right boundary	Find First and Last Position
First true	monotonic condition	First Bad Version
Rotated array	sorted half detection	Search Rotated Sorted Array
Rotated minimum	find pivot/minimum	Find Minimum in Rotated Array
Matrix search	flatten index	Search a 2D Matrix
Peak search	slope logic	Find Peak Element
Pair pattern	index parity	Single Element in Sorted Array

12. Full Problem Walkthrough 1 — LeetCode 704: Binary Search

12.1 Problem Summary

Given a sorted array `nums`, return the index of target.

If not found, return -1.

Example:

`nums = {-1, 0, 3, 5, 9, 12}`

`target = 9`

Output:

4

12.2 How to Identify This Pattern

Problem says:

- sorted array
- search target
- return index

This is exact binary search.

12.3 Brute Force Idea

Scan every element.

Time: $O(n)$

12.4 Optimized Intuition

Use sorted order to eliminate half each time.

12.5 C++ Code

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int left = 0;
        int right = (int)nums.size() - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return -1;
    }
};
```

12.6 Dry Run

nums = {-1, 0, 3, 5, 9, 12}

target = 9

0	5	2	3	target larger, left = 3
3	5	4	9	found

Return 4.

12.7 Correctness Explanation

Because the array is sorted, if $\text{nums}[\text{mid}] < \text{target}$, every value left of or at mid is too small. If $\text{nums}[\text{mid}] > \text{target}$, every value right of or at mid is too large. Thus each step safely discards half.

12.8 Complexity

Time: $O(\log n)$

Space: $O(1)$

12.9 Edge Cases

- empty array
 - one element
 - target at first index
 - target at last index
 - target not present
 - negative values
-

12.10 Common Mistakes

Mistake 1: Using `while (left < right)` for exact search

Can miss the last candidate.

Use:

```
while (left <= right)
```

Mistake 2: Infinite loop from not excluding mid

Correct:

```
left = mid + 1;  
right = mid - 1;
```

12.11 Similar Problems

- Search Insert Position
 - First Bad Version
 - Guess Number Higher or Lower
-

13. Full Problem Walkthrough 2 — LeetCode 35: Search Insert Position

13.1 Problem Summary

Given sorted array and target, return index if target is found.

If not found, return index where it should be inserted.

Example:

```
nums = {1, 3, 5, 6}  
target = 2
```

Output:

```
1
```

13.2 How to Identify This Pattern

Problem asks:

- insertion position
- sorted array

This is lower bound:

first index where $\text{nums}[i] \geq \text{target}$

13.3 C++ Code

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size();

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }

        return left;
    }
};
```

13.4 Dry Run

nums = {1, 3, 5, 6}
target = 2

0	4	2	5	right = 2
0	2	1	3	right = 1
0	1	0	1	left = 1

Return 1.

13.5 Complexity

Time: $O(\log n)$

Space: $O(1)$

13.6 Edge Cases

- target smaller than all
 - target greater than all
 - target exists
 - empty array
 - one element
-

13.7 Common Mistakes

- returning -1 when target not found
 - using exact binary search only
 - accessing `nums[left]` without checking if `left == nums.size()`
-

13.8 Similar Problems

- Lower Bound
 - Find First and Last Position
 - H-Index II
-

14. Full Problem Walkthrough 3 — LeetCode 34: Find First and Last Position of Element in Sorted Array

14.1 Problem Summary

Given sorted array `nums`, find starting and ending position of target.

If not found, return:

{-1, -1}

Example:

`nums = {5, 7, 7, 8, 8, 10}`
`target = 8`

Output:

{3, 4}

14.2 How to Identify This Pattern

Problem asks:

- sorted array
- duplicates
- first and last position

Use binary search twice:

5. first occurrence
 6. last occurrence
-

14.3 C++ Code

```
class Solution {
public:
    int findFirst(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size() - 1;
        int answer = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] >= target) {
                if (nums[mid] == target) {
                    answer = mid;
                }

                right = mid - 1;
            } else {
                left = mid + 1;
            }
        }

        return answer;
    }

    int findLast(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size() - 1;
        int answer = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] <= target) {
                if (nums[mid] == target) {
                    answer = mid;
                }

                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }
};
```

```

    }

    return answer;
}

vector<int> searchRange(vector<int>& nums, int target) {
    return {findFirst(nums, target), findLast(nums, target)};
}
};

```

14.4 Alternative Using STL

```

class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        int first = lower_bound(nums.begin(), nums.end(), target) - nums.begin();
        int last = upper_bound(nums.begin(), nums.end(), target) - nums.begin() - 1;

        if (first < nums.size() && nums[first] == target) {
            return {first, last};
        }

        return {-1, -1};
    }
};

```

14.5 Dry Run

nums = {5, 7, 7, 8, 8, 10}
target = 8

Find first:

0	5	2	7	left = 3
3	5	4	8	answer = 4, right = 3
3	3	3	8	answer = 3, right = 2

First = 3.

Find last:

0	5	2	7	left = 3

left	right	mid	nums[mid]	action
3	5	4	8	answer = 4, left = 5
5	5	5	10	right = 4

Last = 4.

14.6 Complexity

Time: $O(\log n)$

Space: $O(1)$

14.7 Edge Cases

- target not present
 - one occurrence
 - all elements are target
 - empty array
 - target at boundaries
-

14.8 Common Mistakes

- returning first found target
 - not continuing search after finding target
 - incorrect upper_bound adjustment
 - accessing out of bounds when target not found
-

14.9 Similar Problems

- Search Insert Position
 - Count occurrences in sorted array
 - Find first bad version
-

15. Full Problem Walkthrough 4 — LeetCode 744: Find Smallest Letter Greater Than Target

15.1 Problem Summary

Given sorted array of letters and target, return smallest letter greater than target.
Letters wrap around.

Example:

```
letters = {'c', 'f', 'j'}
target = 'a'
```

Output:

```
'c'
```

Example:

```
target = 'j'
```

Output:

```
'c'
```

15.2 How to Identify This Pattern

Problem asks:

first letter greater than target

This is upper bound.

15.3 C++ Code

```
class Solution {
public:
    char nextGreatestLetter(vector<char>& letters, char target) {
        int left = 0;
        int right = letters.size();

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (letters[mid] <= target) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }

        if (left == letters.size()) {
            return letters[0];
        }

        return letters[left];
    }
};
```

15.4 STL Version

```
class Solution {
public:
    char nextGreatestLetter(vector<char>& letters, char target) {
        auto it = upper_bound(letters.begin(), letters.end(), target);
```

```

        if (it == letters.end()) {
            return letters[0];
        }

        return *it;
    }
};

```

15.5 Dry Run

```

letters = {'c', 'f', 'j'}
target = 'f'

```

Need first letter greater than 'f'.

0	3	1	f	left = 2
2	3	2	j	right = 2

Return 'j'.

15.6 Edge Cases

- target smaller than all
- target equal to letter
- target greater than all
- wrap-around
- duplicate letters

15.7 Common Mistake

Using lower bound.

Need strictly greater, so use upper bound.

16. Full Problem Walkthrough 5 — LeetCode 278: First Bad Version

16.1 Problem Summary

Versions are numbered 1 to n.

There is a first bad version.

If a version is bad, every later version is bad.

Find first bad version.

You are given:

```
bool isBadVersion(int version);
```

16.2 How to Identify This Pattern

The condition is monotonic:

good good good bad bad bad

Need first bad.

This is first true binary search.

16.3 C++ Code

```
class Solution {
public:
    int firstBadVersion(int n) {
        int left = 1;
        int right = n;
        int answer = n;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (isBadVersion(mid)) {
                answer = mid;
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        }

        return answer;
    }
};
```

16.4 Code Explanation

If mid is bad:

```
answer = mid;
right = mid - 1;
```

Because first bad could be mid or earlier.

If mid is good:

```
left = mid + 1;
```

Because first bad must be later.

16.5 Complexity

Time: $O(\log n)$ API calls

Space: $O(1)$

16.6 Edge Cases

- first version is bad
 - last version is first bad
 - $n = 1$
-

16.7 Common Mistake

Returning any bad version.

Need first bad, so continue searching left after finding a bad one.

17. Full Problem Walkthrough 6 — LeetCode 33: Search in Rotated Sorted Array

17.1 Problem Summary

Given rotated sorted array with distinct values, search target.

Example:

nums = {4,5,6,7,0,1,2}
target = 0

Output:

4

17.2 How to Identify This Pattern

Problem says:

- originally sorted
- rotated
- search target
- $O(\log n)$

Use modified binary search.

At least one half is sorted.

17.3 Intuition

At each mid:

If left half is sorted:

nums[left] <= nums[mid]

Check if target lies inside left half.

Otherwise, right half is sorted.

17.4 C++ Code

```

class Solution {
public:
    int search(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size() - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) {
                return mid;
            }

            if (nums[left] <= nums[mid]) {
                if (nums[left] <= target && target < nums[mid]) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            } else {
                if (nums[mid] < target && target <= nums[right]) {
                    left = mid + 1;
                } else {
                    right = mid - 1;
                }
            }
        }

        return -1;
    }
};

```

17.5 Dry Run

nums = {4,5,6,7,0,1,2}
target = 0

0	6	3	7	left sorted	target not in 4..7, left = 4
4	6	5	1	left sorted	target in 0..1, right = 4
4	4	4	0	found	return 4

17.6 Complexity

Time: $O(\log n)$

Space: $O(1)$

17.7 Edge Cases

- no rotation
 - one element
 - target absent
 - target at pivot
 - target at boundaries
-

17.8 Important Note About Duplicates

This solution assumes distinct values.

If duplicates exist, identifying sorted half becomes harder.

That is LeetCode 81.

17.9 Common Mistakes

- assuming both halves are sorted
 - using wrong boundary comparisons
 - forgetting no-rotation case
 - infinite loop from wrong pointer update
-

17.10 Similar Problems

- Search in Rotated Sorted Array II
 - Find Minimum in Rotated Sorted Array
 - Find Minimum in Rotated Sorted Array II
-

18. Full Problem Walkthrough 7 — LeetCode 153: Find Minimum in Rotated Sorted Array

18.1 Problem Summary

Given rotated sorted array with distinct elements, return minimum element.

Example:

nums = {3, 4, 5, 1, 2}

Output:

1

18.2 How to Identify This Pattern

Problem asks:

- rotated sorted array

- find minimum
- $O(\log n)$

Minimum is rotation point.

Use binary search comparing `nums[mid]` with `nums[right]`.

18.3 C++ Code

```
class Solution {
public:
    int findMin(vector<int>& nums) {
        int left = 0;
        int right = nums.size() - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] > nums[right]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }

        return nums[left];
    }
};
```

18.4 Why Compare With Right?

If:

`nums[mid] > nums[right]`

Then mid is in the left/larger rotated part.

Minimum must be to the right.

Otherwise:

`nums[mid] <= nums[right]`

Minimum is at mid or to the left.

18.5 Dry Run

`nums = {3,4,5,1,2}`

0	4	2	5	2	left = 3

left	right	mid	nums[mid]	nums[right]	action
3	4	3	1	2	right = 3

Return nums[3] = 1.

18.6 Complexity

Time: $O(\log n)$

Space: $O(1)$

18.7 Edge Cases

- not rotated
- one element
- rotated by one
- minimum at last index
- minimum at first index

18.8 Common Mistake

Using:

```
right = mid - 1;
```

when `nums[mid] <= nums[right]`.

Wrong because mid could be the minimum.

Correct:

```
right = mid;
```

19. Full Problem Walkthrough 8 — LeetCode 74: Search a 2D Matrix

19.1 Problem Summary

Matrix has:

7. each row sorted
8. first integer of each row greater than last integer of previous row

Search target.

Example:

```
matrix = {
  {1, 3, 5, 7},
  {10, 11, 16, 20},
  {23, 30, 34, 60}
}
target = 3
```

Output:

```
true
```

19.2 How to Identify This Pattern

The entire matrix behaves like one sorted 1D array.
Use binary search over virtual indices.

19.3 Index Mapping

If matrix has cols columns:

```
row = index / cols  
col = index % cols
```

19.4 C++ Code

```
class Solution {  
public:  
    bool searchMatrix(vector<vector<int>>& matrix, int target) {  
        int rows = matrix.size();  
        int cols = matrix[0].size();  
  
        int left = 0;  
        int right = rows * cols - 1;  
  
        while (left <= right) {  
            int mid = left + (right - left) / 2;  
  
            int r = mid / cols;  
            int c = mid % cols;  
  
            int value = matrix[r][c];  
  
            if (value == target) {  
                return true;  
            } else if (value < target) {  
                left = mid + 1;  
            } else {  
                right = mid - 1;  
            }  
        }  
  
        return false;  
    }  
};
```

19.5 Dry Run

Search target 16.

Matrix has 3 rows and 4 columns.

Virtual index range:

0 to 11

0	11	5	matrix[1][1]	11	left = 6
6	11	8	matrix[2][0]	23	right = 7
6	7	6	matrix[1][2]	16	found

19.6 Complexity

Time:

$O(\log(\text{rows} * \text{cols}))$

Space: $O(1)$

19.7 Edge Cases

- one row
- one column
- target smaller than all
- target larger than all
- one cell

19.8 Common Mistake

Using this approach on LeetCode 240 matrix.

LeetCode 240 has rows and columns sorted, but not globally flattened sorted.

Different algorithm needed.

20. Full Problem Walkthrough 9 — LeetCode 240: Search a 2D Matrix II

20.1 Problem Summary

Matrix has:

- each row sorted left to right
- each column sorted top to bottom

Search target.

Example:

```
matrix = {  
  {1, 4, 7, 11},
```

```
{2, 5, 8, 12},  
{3, 6, 9, 16}  
}  
target = 5
```

Output:

```
true
```

20.2 How to Identify Pattern

This matrix is not globally sorted as a flattened array.

But it has sorted rows and columns.

Best common approach:

Start at top-right.

20.3 Intuition

Start at:

```
r = 0  
c = cols - 1
```

At top-right:

- everything left is smaller
- everything below is larger

If current value is greater than target, move left.

If current value is smaller than target, move down.

20.4 C++ Code

```
class Solution {  
public:  
    bool searchMatrix(vector<vector<int>>& matrix, int target) {  
        int rows = matrix.size();  
        int cols = matrix[0].size();  
  
        int r = 0;  
        int c = cols - 1;  
  
        while (r < rows && c >= 0) {  
            if (matrix[r][c] == target) {  
                return true;  
            } else if (matrix[r][c] > target) {  
                c--;  
            } else {  
                r++;  
            }  
        }  
    }  
}
```

```
        return false;
    }
};
```

20.5 Is This Binary Search?

Not exactly classic binary search, but it uses sorted structure to eliminate one row or one column each step.

It is often grouped with matrix search patterns.

20.6 Complexity

Time:

$O(\text{rows} + \text{cols})$

Space: $O(1)$

20.7 Common Mistake

Trying to flatten and binary search.

That is invalid unless each row's first element is greater than previous row's last element.

21. Full Problem Walkthrough 10 — LeetCode 162: Find Peak Element

21.1 Problem Summary

A peak element is greater than its neighbors.

Return any peak index.

Example:

`nums = {1, 2, 3, 1}`

Output:

2

21.2 How to Identify Binary Search

Problem asks for a peak in $O(\log n)$.

The array is not sorted, but slope gives direction.

If:

`nums[mid] < nums[mid + 1]`

then a peak exists to the right.

Otherwise, a peak exists at mid or to the left.

21.3 C++ Code

```
class Solution {
public:
    int findPeakElement(vector<int>& nums) {
        int left = 0;
        int right = nums.size() - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] < nums[mid + 1]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }

        return left;
    }
};
```

21.4 Why This Works

If slope goes upward from mid to mid + 1, then moving right must eventually reach a peak.

If slope goes downward, mid or something left can be a peak.

21.5 Complexity

Time: $O(\log n)$

Space: $O(1)$

21.6 Edge Cases

- one element
 - strictly increasing
 - strictly decreasing
 - multiple peaks
-

21.7 Common Mistake

Using mid + 1 when mid could be last.

This template uses:

```
while (left < right)
```

so mid < right, therefore mid + 1 is safe.

22. Full Problem Walkthrough 11 — LeetCode 540: Single Element in a Sorted Array

22.1 Problem Summary

Sorted array where every element appears exactly twice except one element appears once.

Find the single element in $O(\log n)$.

Example:

```
nums = {1,1,2,3,3,4,4,8,8}
```

Output:

```
2
```

22.2 How to Identify Binary Search

Array is sorted.

Pairs follow index pattern.

Before single element:

```
pair starts at even index
```

After single element:

```
pair starts at odd index
```

Use parity.

22.3 C++ Code

```
class Solution {
public:
    int singleNonDuplicate(vector<int>& nums) {
        int left = 0;
        int right = nums.size() - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (mid % 2 == 1) {
                mid--;
            }

            if (nums[mid] == nums[mid + 1]) {
                left = mid + 2;
            } else {
```

```

        right = mid;
    }
}

return nums[left];
}
};

```

22.4 Dry Run

nums = {1,1,2,3,3,4,4,8,8}

0	8	4	3,4 mismatch	right = 4
0	4	2	2,3 mismatch	right = 2
0	2	0	1,1 match	left = 2

Return nums[2] = 2.

22.5 Complexity

Time: $O(\log n)$

Space: $O(1)$

22.6 Edge Cases

- single element array
- unique element at beginning
- unique element at end
- unique element in middle

22.7 Common Mistake

Not forcing mid to even.

The pair comparison becomes inconsistent.

23. Full Problem Walkthrough 12 — LeetCode 69: Sqrt(x)

23.1 Problem Summary

Given non-negative integer x, return integer square root.

That means floor of square root.

Example:

x = 8

Output:

2

because:

$\text{sqrt}(8) = 2.828\dots$

23.2 How to Identify Binary Search

We need largest integer m such that:

$m * m \leq x$

This is last true.

23.3 C++ Code

```
class Solution {
public:
    int mySqrt(int x) {
        int left = 0;
        int right = x;
        int answer = 0;

        while (left <= right) {
            long long mid = left + (right - left) / 2;
            long long square = mid * mid;

            if (square <= x) {
                answer = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return answer;
    }
};
```

23.4 Why long long?

$\text{mid} * \text{mid}$ can overflow int.

Example:

$x = 2147395599$

Use:

$\text{long long square} = \text{mid} * \text{mid};$

23.5 Complexity

Time: $O(\log x)$

Space: $O(1)$

23.6 Edge Cases

- $x = 0$
 - $x = 1$
 - perfect square
 - non-perfect square
 - large x
-

23.7 Similar Problems

- Valid Perfect Square
 - Guess Number Higher or Lower
 - Binary Search on Answer problems
-

24. Full Problem Walkthrough 13 — LeetCode 367: Valid Perfect Square

24.1 Problem Summary

Given positive integer num, return true if it is a perfect square.

Example:

num = 16

Output:

true

Example:

num = 14

Output:

false

24.2 How to Identify Binary Search

Need check if there exists integer m such that:

$m * m == num$

Search range:

1 to num

24.3 C++ Code

```
class Solution {
public:
    bool isPerfectSquare(int num) {
        int left = 1;
        int right = num;

        while (left <= right) {
            long long mid = left + (right - left) / 2;
            long long square = mid * mid;

            if (square == num) {
                return true;
            } else if (square < num) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return false;
    }
};
```

24.4 Complexity

Time: $O(\log \text{num})$

Space: $O(1)$

24.5 Edge Cases

- num = 1
 - large num
 - non-perfect square
 - square near int limit
-

25. Full Problem Walkthrough 14 — LeetCode 275: H-Index II

25.1 Problem Summary

Given sorted array citations, return H-index.

citations is sorted ascending.

H-index is maximum h such that at least h papers have at least h citations.

Example:

citations = {0, 1, 3, 5, 6}

Output:

3

There are 3 papers with at least 3 citations:

3, 5, 6

25.2 How to Identify Binary Search

Sorted array.

At index i, number of papers with at least citations[i] citations is:

$n - i$

Need first index where:

$\text{citations}[i] \geq n - i$

25.3 C++ Code

```
class Solution {
public:
    int hIndex(vector<int>& citations) {
        int n = citations.size();

        int left = 0;
        int right = n - 1;
        int answer = 0;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            int papers = n - mid;

            if (citations[mid] >= papers) {
                answer = papers;
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        }

        return answer;
    }
};
```

25.4 Dry Run

citations = {0, 1, 3, 5, 6}

0	4	2	3	3	answer = 3, search left
0	1	0	0	5	left = 1
1	1	1	1	4	left = 2

Return 3.

25.5 Complexity

Time: $O(\log n)$

Space: $O(1)$

25.6 Edge Cases

- all zero citations
 - all high citations
 - one paper
 - answer 0
-

26. Full Problem Walkthrough 15 — LeetCode 852: Peak Index in a Mountain Array

26.1 Problem Summary

Given mountain array, return peak index.

Mountain array:

- strictly increasing up to peak
- strictly decreasing after peak

Example:

arr = {0, 2, 4, 3, 1}

Output:

2

26.2 How to Identify Binary Search

Mountain array has increasing then decreasing slope.

Compare:

arr[mid] < arr[mid + 1]

If true, peak is to the right.

Else peak is at mid or left.

26.3 C++ Code

```
class Solution {
public:
    int peakIndexInMountainArray(vector<int>& arr) {
        int left = 0;
        int right = arr.size() - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] < arr[mid + 1]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }

        return left;
    }
};
```

26.4 Complexity

Time: $O(\log n)$

Space: $O(1)$

26.5 Similar Problems

- Find Peak Element
 - Find in Mountain Array
 - Bitonic array search
-

27. Mini Problem 1 — Guess Number Higher or Lower

Pattern

Exact binary search with API feedback.

```
class Solution {
public:
    int guessNumber(int n) {
        int left = 1;
```

```

int right = n;

while (left <= right) {
    int mid = left + (right - left) / 2;

    int result = guess(mid);

    if (result == 0) {
        return mid;
    } else if (result == 1) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

return -1;
}
};

```

28. Mini Problem 2 — Count Occurrences in Sorted Array

Use lower and upper bound:

```

int countOccurrences(vector<int>& nums, int target) {
    int left = lower_bound(nums.begin(), nums.end(), target) - nums.begin();
    int right = upper_bound(nums.begin(), nums.end(), target) - nums.begin();

    return right - left;
}

```

If target does not exist, result is 0.

29. Mini Problem 3 — Find First Element Greater Than X

This is upper bound.

```

int firstGreater(vector<int>& nums, int x) {
    int left = 0;
    int right = nums.size();

```

```
while (left < right) {
    int mid = left + (right - left) / 2;

    if (nums[mid] <= x) {
        left = mid + 1;
    } else {
        right = mid;
    }
}

return left;
}
```

30. Mini Problem 4 — Find Last Element Less Than X

Use lower bound minus one.

```
int lastLessThan(vector<int>& nums, int x) {
    int idx = lower_bound(nums.begin(), nums.end(), x) - nums.begin();

    return idx - 1;
}
```

Check:

```
if (idx - 1 >= 0)
```

before accessing.

31. Mini Problem 5 — Search in Infinite Sorted Array Concept

If array size unknown, first expand range:

```
int left = 0;
int right = 1;

while (get(right) < target) {
    left = right + 1;
    right *= 2;
}
```

Then binary search in [left, right].

This is useful in theoretical interviews.

32. Common Edge Cases for Binary Search

32.1 Empty Array

For exact search:

right = -1

Loop does not run.

Return -1.

32.2 One Element

Use:

while (left <= right)

so the one element is checked.

32.3 Target at Boundaries

Test:

- target at index 0
 - target at index n - 1
-

32.4 Target Not Found

Return correct default:

Exact search	-1
Insert position	left
Lower bound	index, possibly n
First occurrence	-1
Last occurrence	-1

32.5 Duplicates

Exact binary search may return any matching index.

Use boundary search for first/last.

32.6 Overflow in Mid

Use:

$\text{left} + (\text{right} - \text{left}) / 2$

32.7 Overflow in Square

Use:

```
long long square = 1LL * mid * mid;
```

32.8 Infinite Loop

Avoid:

```
left = mid;
```

with lower mid.

If using this style, you need upper mid carefully.

For beginners, prefer:

```
left = mid + 1;  
right = mid - 1;
```

or:

```
right = mid
```

in half-open/lower-bound templates.

32.9 Lower Bound Returning n

Before accessing:

```
nums[index]
```

check:

```
index < nums.size()
```

32.10 Rotated Array With Duplicates

Extra care needed.

When:

```
nums[left] == nums[mid] == nums[right]
```

you may not know sorted half.

Usually shrink boundaries carefully.

33. Common Mistakes and Wrong Code

Mistake 1 — Wrong Loop Condition

Wrong for exact search:

```
while (left < right)
```

May skip last candidate.

Correct:

```
while (left <= right)
```

Mistake 2 — Not Excluding Mid

Wrong:

```
left = mid;
```

Can infinite loop.

Correct:

```
left = mid + 1;
```

Mistake 3 — Wrong Lower Bound Update

For lower bound, if `nums[mid] >= target`, `mid` could be answer.

Correct:

```
right = mid;
```

not:

```
right = mid - 1;
```

Mistake 4 — Returning mid After Loop

`mid` may be stale.

Return:

```
left
```

or tracked answer.

Mistake 5 — Forgetting Duplicates

For first occurrence, keep searching left after finding target.

For last occurrence, keep searching right.

Mistake 6 — Binary Searching Non-Monotonic Data

Binary search only works if you can eliminate half.

Mistake 7 — Sorting When Order Matters

Do not sort if original order has meaning.

34. Interview Explanation Templates

34.1 Exact Search

Since the array is sorted, I compare the target with the middle element. If target is smaller, I discard the right half. If target is larger, I discard the left half. This halves the search space each step.

34.2 Lower Bound

I want the first index where the value is at least target. If middle is smaller than target, it cannot be the answer, so I move left to mid + 1. Otherwise, mid could be the answer, so I keep it by moving right to mid.

34.3 First and Last Occurrence

A normal binary search may find any occurrence. For first occurrence, when I find target I record it and continue searching left. For last occurrence, I record it and continue searching right.

34.4 Rotated Array

In a rotated sorted array, at least one half around the middle is sorted. I identify the sorted half and check if the target lies inside it. If yes, I search that half; otherwise I search the other half.

34.5 Matrix Search

Since the matrix is globally sorted when flattened, I binary search over virtual indices and map each index to row and column using division and modulo.

35. Complexity Patterns

Exact binary search	$O(\log n)$	$O(1)$
Lower bound	$O(\log n)$	$O(1)$
Upper bound	$O(\log n)$	$O(1)$
First/last occurrence	$O(\log n)$	$O(1)$
First bad version	$O(\log n)$	$O(1)$
Rotated search	$O(\log n)$	$O(1)$
Find min rotated	$O(\log n)$	$O(1)$
Search 2D Matrix	$O(\log(mn))$	$O(1)$
Search 2D Matrix II	$O(m + n)$	$O(1)$
Peak element	$O(\log n)$	$O(1)$
Sqrt(x)	$O(\log x)$	$O(1)$

36. Time-Boxed Practice Guidance

Easy Binary Search

Target:

5–15 minutes

Examples:

- Binary Search
- Search Insert Position
- Guess Number
- Valid Perfect Square

Medium Boundary / Special Search

Target:

20–35 minutes

Examples:

- Find First and Last Position
- Search Rotated Sorted Array
- Find Minimum in Rotated Sorted Array
- Search a 2D Matrix
- Single Element in Sorted Array

Harder Binary Search Thinking

Target:

35–50 minutes

Examples:

- Search in Rotated Sorted Array II
- Find Minimum in Rotated Sorted Array II
- Median of Two Sorted Arrays
- Find in Mountain Array

37. Expanded Practice Set

37.1 Must-Do Easy

- 1.1. Binary Search
- 2.2. Search Insert Position
- 3.3. First Bad Version
- 4.4. Guess Number Higher or Lower
- 5.5. Sqrt(x)
- 6.6. Valid Perfect Square
- 7.7. Find Smallest Letter Greater Than Target

37.2 Must-Do Medium

- 8.8. Find First and Last Position of Element in Sorted Array
- 9.9. Search in Rotated Sorted Array
- 10.10. Find Minimum in Rotated Sorted Array
- 11.11. Search a 2D Matrix
- 12.12. Search a 2D Matrix II
- 13.13. Find Peak Element
- 14.14. Single Element in a Sorted Array

- 15.15. H-Index II
 - 16.16. Peak Index in a Mountain Array
 - 17.17. Find K Closest Elements
-

37.3 Strong Interview Practice

- 18.18. Search in Rotated Sorted Array II
 - 19.19. Find Minimum in Rotated Sorted Array II
 - 20.20. Median of Two Sorted Arrays
 - 21.21. Find in Mountain Array
 - 22.22. Kth Smallest Element in a Sorted Matrix
 - 23.23. Longest Increasing Subsequence, binary search variant
 - 24.24. Find the Duplicate Number, binary search on value variant
 - 25.25. Split Array Largest Sum, next chapter
 - 26.26. Koko Eating Bananas, next chapter
-

37.4 Hard / Stretch

- 27.27. Median of Two Sorted Arrays
- 28.28. Find in Mountain Array
- 29.29. Find K-th Smallest Pair Distance
- 30.30. Kth Smallest Number in Multiplication Table
- 31.31. K-th Smallest Prime Fraction
- 32.32. Nth Magical Number

Some of these are really **Binary Search on Answer**, which comes next.

38. Suggested Study Order

Step 1 — Exact Search

- 9. Binary Search
- 10. Guess Number Higher or Lower
- 11. Valid Perfect Square

Goal:

Master basic left/right/mid.

Step 2 — Lower / Upper Bound

- 12. Search Insert Position
- 13. Find Smallest Letter Greater Than Target
- 14. Find First and Last Position

Goal:

Master boundaries.

Step 3 — Rotated Arrays

- 15. Search in Rotated Sorted Array
- 16. Find Minimum in Rotated Sorted Array
- 17. Search in Rotated Sorted Array II
- 18. Find Minimum in Rotated Sorted Array II

Goal:

Understand sorted-half logic.

Step 4 — Matrix and Special Structures

- 19. Search a 2D Matrix
- 20. Search a 2D Matrix II
- 21. Find Peak Element
- 22. Single Element in Sorted Array
- 23. H-Index II

Goal:

Use binary search beyond simple arrays.

Step 5 — Advanced Boundary Thinking

- 24. Median of Two Sorted Arrays
- 25. Find in Mountain Array
- 26. Kth Smallest Element in Sorted Matrix

Goal:

Prepare for hard binary search.

39. Pattern Decision Table

find target in sorted array	exact search
insertion index	lower bound
first occurrence	left boundary
last occurrence	right boundary
first greater than target	upper bound
first bad version	first true
rotated sorted array	sorted-half search
minimum in rotated array	compare mid with right
globally sorted matrix	flattened binary search
row/column sorted matrix	top-right elimination
peak element	slope binary search
single element among pairs	parity binary search
integer square root	last $mid * mid \leq x$
h-index sorted citations	first feasible index

40. Final Cheat Sheet

Exact Search

```
int left = 0;
int right = nums.size() - 1;

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (nums[mid] == target) {
        return mid;
    } else if (nums[mid] < target) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

return -1;
```

Lower Bound

```
int left = 0;
int right = nums.size();

while (left < right) {
    int mid = left + (right - left) / 2;

    if (nums[mid] < target) {
        left = mid + 1;
    } else {
        right = mid;
    }
}

return left;
```

Upper Bound

```
int left = 0;
int right = nums.size();

while (left < right) {
    int mid = left + (right - left) / 2;
```

```
    if (nums[mid] <= target) {
        left = mid + 1;
    } else {
        right = mid;
    }
}

return left;
```

First Occurrence

```
int answer = -1;

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (nums[mid] >= target) {
        if (nums[mid] == target) {
            answer = mid;
        }

        right = mid - 1;
    } else {
        left = mid + 1;
    }
}
```

Last Occurrence

```
int answer = -1;

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (nums[mid] <= target) {
        if (nums[mid] == target) {
            answer = mid;
        }

        left = mid + 1;
    } else {
        right = mid - 1;
    }
}
```

```
}  
}
```

Rotated Search

```
if (nums[left] <= nums[mid]) {  
    if (nums[left] <= target && target < nums[mid]) {  
        right = mid - 1;  
    } else {  
        left = mid + 1;  
    }  
} else {  
    if (nums[mid] < target && target <= nums[right]) {  
        left = mid + 1;  
    } else {  
        right = mid - 1;  
    }  
}
```

Find Minimum Rotated

```
while (left < right) {  
    int mid = left + (right - left) / 2;  
  
    if (nums[mid] > nums[right]) {  
        left = mid + 1;  
    } else {  
        right = mid;  
    }  
}  
  
return nums[left];
```

2D Matrix Mapping

```
int r = mid / cols;  
int c = mid % cols;
```

Peak Search

```
while (left < right) {  
    int mid = left + (right - left) / 2;
```

```
if (nums[mid] < nums[mid + 1]) {  
    left = mid + 1;  
} else {  
    right = mid;  
}  
}  
  
return left;
```

41. What to Master Before Chapter 10

Before moving to Binary Search on Answer, make sure you can:

- Write exact binary search from memory
- Explain why sorted order allows half elimination
- Use safe mid formula
- Distinguish $\text{left} \leq \text{right}$ from $\text{left} < \text{right}$
- Implement lower bound
- Implement upper bound
- Find first occurrence
- Find last occurrence
- Handle duplicates
- Return insertion position
- Search rotated sorted array
- Find minimum in rotated sorted array
- Search a flattened 2D matrix
- Solve peak element
- Use long long for square calculations
- Avoid infinite loops
- Know when binary search does not apply

42. Core Takeaway

Binary Search is about eliminating half the search space using structure.

The key question is:

Can I decide which half cannot contain the answer?

If yes, binary search may apply.

There are three major ideas in this chapter:

27. **Exact search**

Find target directly.

28. **Boundary search**

Find first/last index satisfying a condition.

29. **Special sorted structures**

Rotated arrays, matrices, peaks, and pair patterns.

The most important formulas are:

$\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$

$\text{lower_bound} = \text{first index where } \text{nums}[i] \geq \text{target}$

$\text{upper_bound} = \text{first index where } \text{nums}[i] > \text{target}$

Binary search is not about memorizing one template.

It is about defining:

- search space
- condition
- direction
- return value

Once those are clear, the code becomes much easier.
